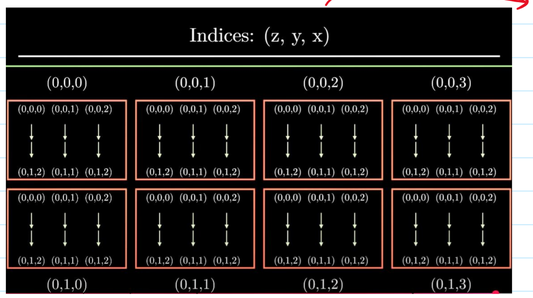
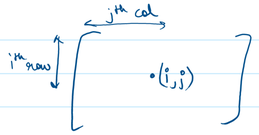
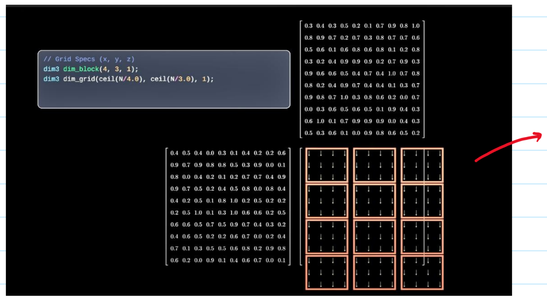
# NAIVE

04 June 2025 16:46




this is how the indexing of block & thread is done (z, y, x)



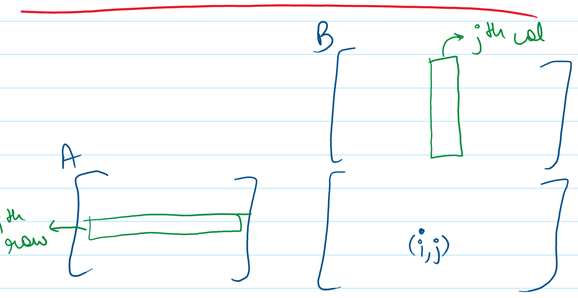

here we have blocks with
X dim → 4
Y dim → 3

so that makes block_dim (4, 3, 1)

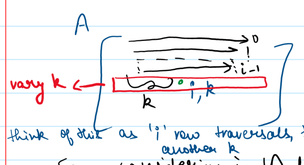
* this is the dimensions & not indexing hence, (x, y, z) format

Now we know, that if we fix block-Dim then we can find grid-dim using

$$\text{grid_dim} \left( \frac{N + \text{block_dim.x} - 1}{\text{block_dim.x}}, \frac{N + \text{block_dim.y} - 1}{\text{block_dim.y}}, 1 \right)$$




$(i,j)^{th}$ element in C is formed by dot product of $i^{th}$ row of A and $j^{th}$ col of B

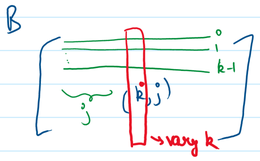



think of this as 'i' row traversals, then another k

So considering in 1D format

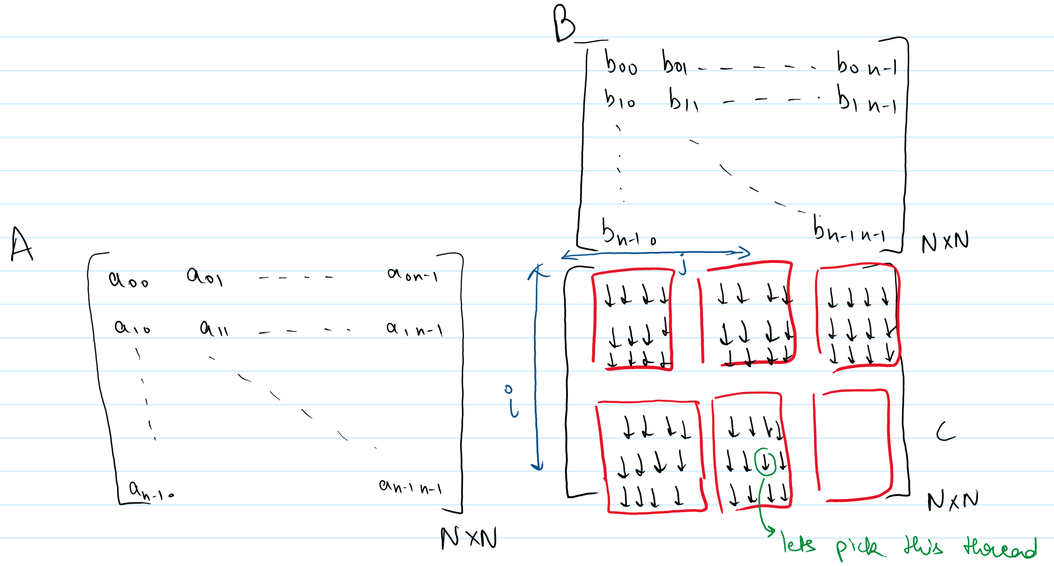
A[i][k] = A[i x N + k]

- now we can vary 'k' & get entire ith row
- then for next row, just increment 'i'




consider this as 'k' row traversals & then iteration next row using 'k'

So B[k][j] = B[k x N + j]




and see what element of C will it be working for

$i = \text{blockDim.y} * \text{blockIdx.y} + \text{threadIdx.y}$
$j = \text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x}$

we know blockDim.y = 3
blockDim.x = 4

& the thread is in block (0, 1, 1) (z y x)

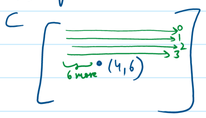
so blockIdx.y = 1
blockIdx.x = 1

& locally in block thread index is (0, 1, 2) (z y x)

threadIdx.y = 1
threadIdx.x = 2

so $i = 3 \times 1 + 1 \Rightarrow 4$
$j = 4 \times 1 + 2 \quad 6$

$i = 4$
$j = 6$

So it will be working for C[i][j] or C[4][6]

i.e. C[i * N + j] in 1D format